Multi-Source Candidate Data Transformer — Technical Design

Rama Lokesh Reddy Penumallu · rlpreddy565@gmail.com · Eightfold Engineering Intern Assignment

Pipeline / Step Breakdown

```
parse → extract (per-source, into FieldValue list) → merge (conflict resolution + confidence) → project (apply runtime config) → validate
```

Extractors never write to the canonical profile directly — each emits a flat list of `FieldValue(field_name, value, source, method, confidence)` objects. Keeping extraction and merging separate means I can test each source's parsing in isolation, and the merge engine doesn't need to know anything about CSV columns or GitHub's API shape.

Canonical Output Schema & Normalization

- **Phones:** E.164 (`+91XXXXXXXXXX`). Numbers under 10 digits return `null` rather than guess a country code from a fragment.
- **Dates:** `YYYY-MM`. Handles ISO prefixes, "Mon YYYY", and year-only (defaults to `-01`, lower confidence implied by caller).
- **Skills:** small curated alias map (`"js"` / `"reactjs"` → `"JavaScript"` / `"React"`) rather than a full taxonomy — sufficient for this scope, explicitly not enterprise-grade.
- **Country:** ISO-3166 alpha-2, inferred only when a source states it explicitly.

Merge / Conflict-Resolution Policy

Core idea: confidence is per-field-per-source, not per-source. A recruiter CSV is authoritative for phone/email (a human typed it deliberately) but weak for skills (rarely updated). GitHub is the inverse — untrustworthy for "current company" but strong, if indirect, evidence for technical skills via repo languages. This is encoded as a small prior table in `schema.py` (e.g. `emails: recruiter_csv=0.9, github=0.4`) rather than one trust score per source.

- **Single-value fields** (name, headline): highest-confidence `FieldValue` wins outright. Exact ties are broken by a fixed source-priority list — never randomly — so the same inputs always produce the same output.
- **Multi-value fields** (emails, phones, skills): union + dedupe. Two emails from two sources are two real facts, not a conflict to resolve down to one.
- **Object-list fields** (experience): deduplicated by `(company, title)`; on collision, the entry with more populated sub-fields wins, but the loser is still appended to provenance — nothing is silently thrown away.
- **GitHub-inferred skills are explicitly discounted** (−0.15) relative to a self-declared skill, because "has a repo containing some JS" is weaker evidence than "the candidate listed this skill."

`overall_confidence` = mean of confidences of values that actually *won* a slot — not an average across every candidate value, which would unfairly penalize profiles with more disagreeing sources even when the merge resolved them correctly.

Runtime Custom-Output Config

The projection layer (`project.py`) reads the canonical profile dict and reshapes it per a JSON config — field subset, renaming via `from` paths (including `skills[].name` list-mapping and `emails[0]` indexing), per-field `normalize` rules, and an `on_missing` policy (`null` / `omit` / `error`). It never mutates the canonical record — the canonical profile is computed once by `merge.py` and projected as many times as needed, cheaply and consistently. Validation (`validate.py`) runs after projection and, for `on_missing="error"`, raises with the specific missing field named rather than failing silently.

Edge Cases Handled

- **Malformed/garbage CSV** (wrong columns): extractor returns an empty list, not an exception — verified by test.
- **Missing source entirely:** pipeline produces a valid partial profile; absent fields are `null`, never invented.
- **Conflicting names/titles across sources:** resolved deterministically by confidence + priority, with the discarded value kept in provenance for audit.
- **Duplicate experience entries** with different completeness (e.g. ATS has dates, recruiter CSV doesn't): richer entry wins by field-count, not by source seniority alone.
- **Required field genuinely missing** under a custom config with `on_missing="error"`: raises a specific, named error rather than returning a half-correct object.

What I Deliberately Left Out Under Time Pressure

LinkedIn and resume-PDF extractors are designed in the schema (priors are already defined for both) but not implemented — I prioritized making the merge and projection engine genuinely correct (with tests) over adding a third or fourth source type with shallow logic. A real phone-number library (e.g. `phonenumbers`) would replace my regex-based E.164 normalizer in production; I default to India's country code when no `+` prefix is present, which is a reasonable assumption for this exercise but not a generally correct one. Resume/DOCX parsing would need a PDF text-extraction layer plus an LLM or NER pass to structure the prose — out of scope for the time available, and I'd rather state that honestly than fake a shallow regex-based resume parser that looks complete but isn't trustworthy.

Full implementation, tests, and sample I/O: see accompanying GitHub repository and README.